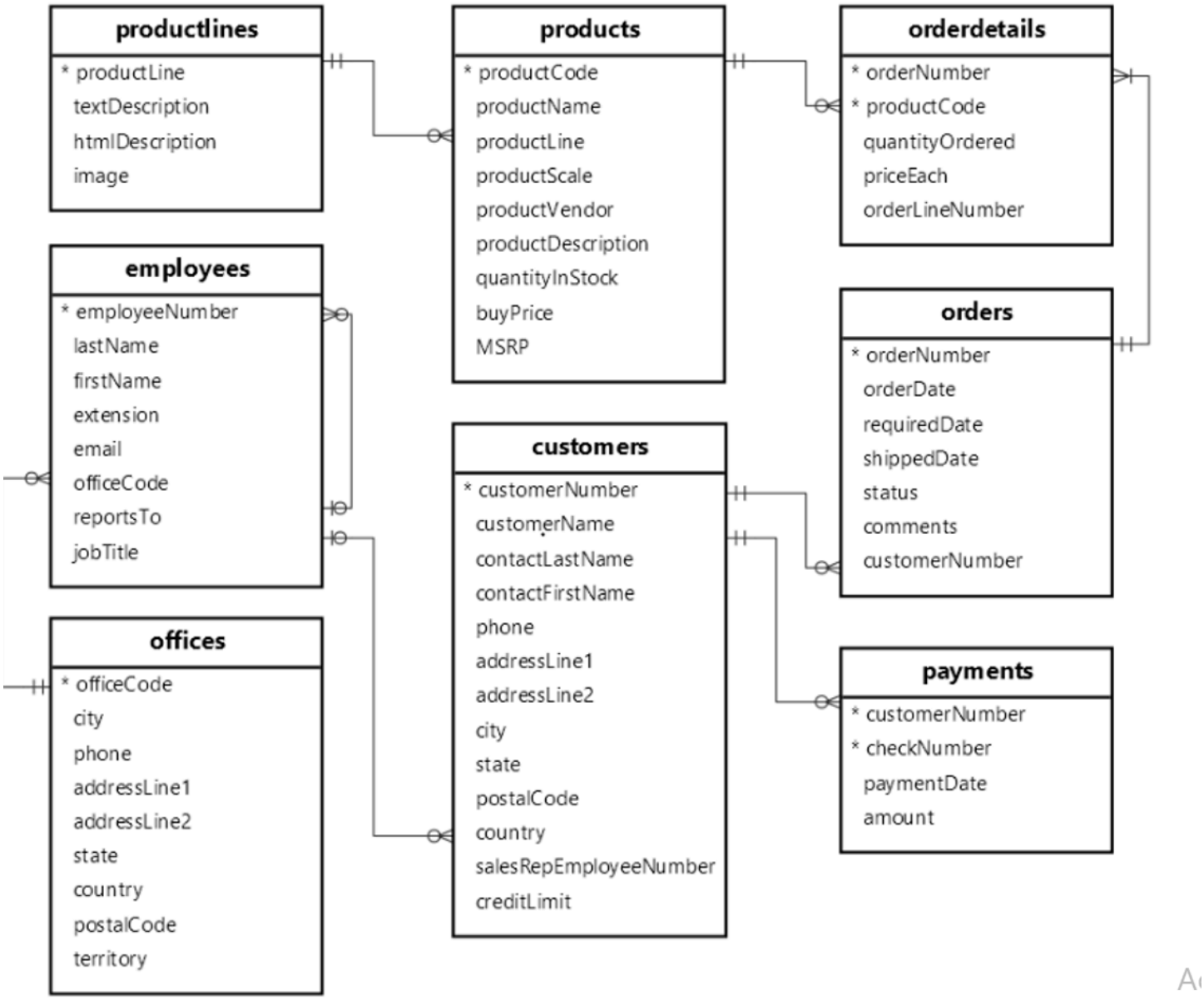

The fields above mentioned in the * are the Primary keys of that table
eg: *officeCode, *CustomerNumber etc.

. **CREATE TABLES using the below structure and insert data into it**

```
-- OFFICES
CREATE TABLE offices (
   officeCode VARCHAR(10) PRIMARY KEY,
   city VARCHAR(50),
   phone VARCHAR(50),
   addressLine1 VARCHAR(100),
   addressLine2 VARCHAR(100),
   state VARCHAR(50),
   country VARCHAR(50),
```

```
    postalCode VARCHAR(15),
    territory VARCHAR(50)
);

-- EMPLOYEES
CREATE TABLE employees (
    employeeNumber INT PRIMARY KEY,
    lastName VARCHAR(50),
    firstName VARCHAR(50),
    extension VARCHAR(10),
    email VARCHAR(100),
    officeCode VARCHAR(10),
    reportsTo INT,
    jobTitle VARCHAR(50),
    FOREIGN KEY (officeCode) REFERENCES offices(officeCode),
    FOREIGN KEY (reportsTo) REFERENCES employees(employeeNumber)
);

-- CUSTOMERS
CREATE TABLE customers (
    customerNumber INT PRIMARY KEY,
    customerName VARCHAR(100),
    contactLastName VARCHAR(50),
    contactFirstName VARCHAR(50),
    phone VARCHAR(50),
    addressLine1 VARCHAR(100),
    addressLine2 VARCHAR(100),
    city VARCHAR(50),
    state VARCHAR(50),
    postalCode VARCHAR(15),
    country VARCHAR(50),
    salesRepEmployeeNumber INT,
    creditLimit DECIMAL(10,2),
    FOREIGN KEY (salesRepEmployeeNumber) REFERENCES employees(employeeNumber)
);

-- PRODUCTLINES
CREATE TABLE productlines (
    productLine VARCHAR(50) PRIMARY KEY,
    textDescription TEXT,
    htmlDescription TEXT,
    image TEXT
);

-- PRODUCTS
CREATE TABLE products (
```

```
    productCode VARCHAR(15) PRIMARY KEY,
    productName VARCHAR(100),
    productLine VARCHAR(50),
    productScale VARCHAR(10),
    productVendor VARCHAR(50),
    productDescription TEXT,
    quantityInStock INT,
    buyPrice DECIMAL(10,2),
    MSRP DECIMAL(10,2),
    FOREIGN KEY (productLine) REFERENCES productlines(productLine)
);

-- ORDERS
CREATE TABLE orders (
    orderNumber INT PRIMARY KEY,
    orderDate DATE,
    requiredDate DATE,
    shippedDate DATE,
    status VARCHAR(20),
    comments TEXT,
    customerNumber INT,
    FOREIGN KEY (customerNumber) REFERENCES customers(customerNumber)
);

-- ORDERDETAILS
CREATE TABLE orderdetails (
    orderNumber INT,
    productCode VARCHAR(15),
    quantityOrdered INT,
    priceEach DECIMAL(10,2),
    orderLineNumber INT,
    PRIMARY KEY (orderNumber, productCode),
    FOREIGN KEY (orderNumber) REFERENCES orders(orderNumber),
    FOREIGN KEY (productCode) REFERENCES products(productCode)
);

-- PAYMENTS
CREATE TABLE payments (
    customerNumber INT,
    checkNumber VARCHAR(50),
    paymentDate DATE,
    amount DECIMAL(10,2),
    PRIMARY KEY (customerNumber, checkNumber),
    FOREIGN KEY (customerNumber) REFERENCES customers(customerNumber)
);
```

INSERT SAMPLE DATA

Offices

```
INSERT INTO offices VALUES
('1', 'New York', '1234567890', '5th Ave', NULL, 'NY', 'USA', '10001', 'NA'),
('2', 'London', '9876543210', 'Baker St', NULL, NULL, 'UK', 'NW1', 'EMEA');
```

Employees

```
INSERT INTO employees VALUES
(1001, 'Smith', 'John', 'x101', 'john@company.com', '1', NULL, 'Manager'),
(1002, 'Doe', 'Jane', 'x102', 'jane@company.com', '1', 1001, 'Sales Rep'),
(1003, 'Brown', 'Mike', 'x103', 'mike@company.com', '2', 1001, 'Sales Rep');
```

Customers

```
INSERT INTO customers VALUES
(2001, 'ABC Corp', 'Lee', 'Chris', '1111111111', 'Street 1', NULL, 'NYC', 'NY', '10001', 'USA', 1002, 50000),
(2002, 'XYZ Ltd', 'Kim', 'Pat', '2222222222', 'Street 2', NULL, 'London', NULL, 'NW1', 'UK', 1003, 70000),
(2003, 'Global Inc', 'Singh', 'Raj', '3333333333', 'Street 3', NULL, 'Delhi', NULL, '110001', 'India', 1002, 60000);
```

Product Lines

```
INSERT INTO productlines VALUES
('Cars', 'Car models', NULL, NULL),
('Bikes', 'Bike models', NULL, NULL);
```

Products

```
INSERT INTO products VALUES
('P101', 'Toyota Camry', 'Cars', '1:18', 'Toyota', 'Sedan model', 100, 20.00, 30.00),
('P102', 'Honda Civic', 'Cars', '1:18', 'Honda', 'Compact model', 150, 18.00, 28.00),
('P201', 'Yamaha Bike', 'Bikes', '1:12', 'Yamaha', 'Sport bike', 200, 10.00, 15.00);
```

Orders

```
INSERT INTO orders VALUES
(3001, '2024-01-01', '2024-01-05', '2024-01-03', 'Shipped', NULL, 2001),
(3002, '2024-02-01', '2024-02-05', '2024-02-04', 'Shipped', NULL, 2002),
(3003, '2024-03-01', '2024-03-06', NULL, 'Pending', NULL, 2003);
```

Order Details

```
INSERT INTO orderdetails VALUES
(3001, 'P101', 10, 30.00, 1),
```

(3001, 'P201', 5, 15.00, 2),
(3002, 'P102', 7, 28.00, 1),
(3003, 'P101', 3, 30.00, 1);

Payments

INSERT INTO payments VALUES
(2001, 'CHK001', '2024-01-10', 300.00),
(2002, 'CHK002', '2024-02-10', 500.00),
(2003, 'CHK003', '2024-03-10', 150.00);

EASY LEVEL (1–20)

1. List all customers ordered by customerName.

```
ysql> select * from customers order by customerName;
----------------+--------------+-----------------+------------------+------------+--------------+--------------+--------+-------+------------+---------+---
--------------------+-------------+
 customerNumber | customerName | contactLastName | contactFirstName | phone      | addressLine1 | addressLine2 | city   | state | postalCode | country | sa
esRepEmployeeNumber | creditLimit |
----------------+--------------+-----------------+------------------+------------+--------------+--------------+--------+-------+------------+---------+---
--------------------+-------------+
           2001 | ABC Corp     | Lee             | Chris            | 1111111111 | Street 1     | NULL         | NYC    | NY    | 10001      | USA     |
               1002 |    50000.00 |
           2003 | Global Inc   | Singh           | Raj              | 3333333333 | Street 3     | NULL         | Delhi  | NULL  | 110001     | India   |
               1002 |    60000.00 |
           2002 | XYZ Ltd      | Kim             | Pat              | 2222222222 | Street 2     | NULL         | London | NULL  | NW1        | UK      |
               1003 |    70000.00 |
----------------+--------------+-----------------+------------------+------------+--------------+--------------+--------+-------+------------+---------+---
--------------------+-------------+
 rows in set (0.00 sec)
```

2. Display all products ordered by buyPrice descending.

```
mysql> select * from products order by buyPrice desc;
+-------------+--------------+-------------+--------------+---------------+--------------------+----------------+----------+-------+
| productCode | productName  | productLine | productScale | productVendor | productDescription | quantityInStock | buyPrice | MSRP  |
+-------------+--------------+-------------+--------------+---------------+--------------------+----------------+----------+-------+
| P101        | Toyota Camry | Cars        | 1:18         | Toyota        | Sedan model        |             100 |    20.00 | 30.00 |
| P102        | Honda Civic  | Cars        | 1:18         | Honda         | Compact model      |             150 |    18.00 | 28.00 |
| P201        | Yamaha Bike  | Bikes       | 1:12         | Yamaha        | Sport bike         |             200 |    10.00 | 15.00 |
+-------------+--------------+-------------+--------------+---------------+--------------------+----------------+----------+-------+
3 rows in set (0.02 sec)
```

3. Show all employees ordered by lastName.

```
mysql> select * from employeess order by lastName;
+----------------+----------+-----------+-----------+------------------+------------+-----------+-----------+
| employeeNumber | lastName | firstName | extension | email            | officeCode | reportsTo | jobTitle  |
+----------------+----------+-----------+-----------+------------------+------------+-----------+-----------+
|           1003 | Brown    | mike      | x103      | mike@company.com | 2          |      1001 | sales rep |
|           1002 | Doe      | jane      | x102      | jane@company.com | 1          |      1001 | sales rep |
|           1001 | smith    | john      | x101      | john@company.com | 1          |      NULL | manager   |
+----------------+----------+-----------+-----------+------------------+------------+-----------+-----------+
3 rows in set (0.02 sec)
```

4. List all offices sorted by country.

```
mysql> select * from offices order by country;
+------------+----------+------------+--------------+--------------+-------+---------+------------+-----------+
| officeCode | city     | phone      | addressLine1 | addressLine2 | state | country | postalCode | territory |
+------------+----------+------------+--------------+--------------+-------+---------+------------+-----------+
| 2          | london   | 9876543210 | baker St     | NULL         | NULL  | UK      | NW1        | EMEA      |
| 1          | new york | 1234567890 | 5th Ave      | NULL         | NY    | USA     | 10001      | NA        |
+------------+----------+------------+--------------+--------------+-------+---------+------------+-----------+
2 rows in set (0.00 sec)
```

5. Get all orders sorted by orderDate descending.

```
mysql> select * from orders order by orderDate desc;
+-------------+------------+--------------+-------------+---------+----------+----------------+
| orderNumber | orderDate  | requiredDate | shippedDate | status  | comments | customerNumber |
+-------------+------------+--------------+-------------+---------+----------+----------------+
|        3003 | 2024-03-01 | 2024-03-06   | NULL        | Pending | NULL     |           2003 |
|        3002 | 2024-02-01 | 2024-02-05   | 2024-02-04  | Shipped | NULL     |           2002 |
|        3001 | 2024-01-01 | 2024-01-05   | 2024-01-03  | Shipped | NULL     |           2001 |
+-------------+------------+--------------+-------------+---------+----------+----------------+
3 rows in set (0.00 sec)
```

6. Show customer names with their sales representative (employee name).

```
mysql> select c.customerName,concat(e.firstName,' ',e.lastName) as salesRepName from customers c join employeess e on c.
salesRepEmployeeNumber = e.employeeNumber;
+--------------+--------------+
| customerName | salesRepName |
+--------------+--------------+
| ABC Corp     | jane Doe     |
| Global Inc   | jane Doe     |
| XYZ Ltd      | mike Brown   |
+--------------+--------------+
3 rows in set (0.03 sec)
```

7. List all products with their product line.

```
mysql> select productName,productLine from products;
+---------------+-------------+
| productName   | productLine |
+---------------+-------------+
| Toyota Camry  | Cars        |
| Honda Civic   | Cars        |
| Yamaha Bike   | Bikes       |
+---------------+-------------+
3 rows in set (0.00 sec)
```

8. Display orders with customer names.

```
mysql> select o.orderNumber ,c.customerName from orders o join customers c on o.customerNumber = c.customerNumber;
+-------------+--------------+
| orderNumber | customerName |
+-------------+--------------+
        3001 | ABC Corp     |
        3002 | XYZ Ltd      |
        3003 | Global Inc   |
+-------------+--------------+
3 rows in set (0.00 sec)
```

9. Show order details with product names.

```
mysql> select od.orderNumber ,p.productName ,od.priceEach from orderdetails od join products p on od.productCode = p.pro
ductCode;
+-------------+--------------+-----------+
| orderNumber | productName  | priceEach |
+-------------+--------------+-----------+
        3001 | Toyota Camry |     30.00 |
        3001 | Yamaha Bike  |     15.00 |
        3002 | Honda Civic  |     28.00 |
        3003 | Toyota Camry |     30.00 |
+-------------+--------------+-----------+
```

10. List payments with customer names.

```
mysql> select c.customerName , p.amount from payments p join customers c on p.customerNumber = c.customerNumber
+--------------+--------+
| customerName | amount |
+--------------+--------+
| ABC Corp     | 300.00 |
| XYZ Ltd      | 500.00 |
| Global Inc   | 150.00 |
+--------------+--------+
3 rows in set (0.00 sec)
```

11. Count total number of customers.

```
mysql> select count(*) as totlCustomers from customers;
+---------------+
| totlCustomers |
+---------------+
|             3 |
+---------------+
1 row in set (0.01 sec)
```

12. Count total number of orders.

```
mysql> select count(*) as totalOrders from orders;
+-------------+
| totalOrders |
+-------------+
|           3 |
+-------------+
1 row in set (0.00 sec)
```

13. Find average buyPrice of products.

```
mysql> select avg(buyPrice) as avg_buy_price from products;
+---------------+
| avg_buy_price |
+---------------+
|     16.000000 |
+---------------+
1 row in set (0.00 sec)
```

14. Find maximum MSRP from products.

```
mysql> select max(MSRP) as max_msrp from products;
+----------+
| max_msrp |
+----------+
|    30.00 |
+----------+
1 row in set (0.02 sec)
```

15. Find total number of employees.

```
mysql> select count(*) as total_employees from employeess;
+-----------------+
| total_employees |
+-----------------+
|               3 |
+-----------------+
1 row in set (0.00 sec)
```

16. Count customers per country.

```
mysql> select country , count(*) as customerCount from customers group by country order by customerCount desc;
+---------+---------------+
| country | customerCount |
+---------+---------------+
| India   |             1 |
| USA     |             1 |
| UK      |             1 |
+---------+---------------+
3 rows in set (0.01 sec)
```

17. Count orders per status.

```
mysql> select status,count(*) as totalorders from orders group by status order by totalOrders desc;
+---------+-------------+
| status  | totalorders |
+---------+-------------+
| Shipped |           2 |
| Pending |           1 |
+---------+-------------+
2 rows in set (0.00 sec)
```

18. Count products per product line

```
mysql> select productLine, count(*) as total_products from products group by productLine order by total_products desc;
+-------------+----------------+
| productLine | total_products |
+-------------+----------------+
| Cars        |              2 |
| Bikes       |              1 |
+-------------+----------------+
2 rows in set (0.00 sec)
```

.

19. Find average credit limit per country.

```
mysql> select country , avg(creditLimit) as avg_credit_limit from customers group by country order by avg_credit_limit d
esc;
+---------+------------------+
| country | avg_credit_limit |
+---------+------------------+
| UK      |     70000.000000 |
| India   |     60000.000000 |
| USA     |     50000.000000 |
+---------+------------------+
3 rows in set (0.00 sec)
```

20. Count employees per office.

```
mysql> select officeCode,count(*) as totalEmployees from employeess group by officeCode;
+------------+----------------+
| officeCode | totalEmployees |
+------------+----------------+
| 1          |              2 |
| 2          |              1 |
+------------+----------------+
2 rows in set (0.00 sec)
```

MEDIUM LEVEL (21–50)

21. Count number of orders per customer.

```
mysql> select c.customerName , count(o.orderNumber) as totalOrders from customers c join orders o  on c.customerNumber=
o.customerNumber group by c.customerNumber ,c.customerName order by totalOrders desc;
+--------------+------------+
| customerName | totalOrders |
+--------------+------------+
| Global Inc   |          1 |
| ABC Corp     |          1 |
| XYZ Ltd      |          1 |
+--------------+------------+
3 rows in set (0.02 sec)
```

22. Find total payment amount per customer.

```
mysql> select c.customerName,sum(p.amount) as totalPaid from customers c join payments p on c.customerNumber =p.customer
Number group by c.customerNumber, c.customerName order by totalPaid desc;
+--------------+----------+
| customerName | totalPaid |
+--------------+----------+
| XYZ Ltd      |   500.00 |
| ABC Corp     |   300.00 |
| Global Inc   |   150.00 |
+--------------+----------+
3 rows in set (0.00 sec)
```

23. List total quantity ordered per product.

```
mysql> SELECT p.productName, SUM(od.quantityOrdered) AS totalQty FROM products p JOIN orderdetails od ON p.productCode =
 od.productCode GROUP BY p.productCode, p.productName ORDER BY totalQty DESC;
+--------------+----------+
| productName  | totalQty |
+--------------+----------+
| Toyota Camry |       13 |
| Honda Civic  |        7 |
| Yamaha Bike  |        5 |
+--------------+----------+
3 rows in set (0.00 sec)
```

24. Show total revenue per product (quantityOrdered * priceEach).

```
mysql> SELECT p.productName,
    ->        SUM(od.quantityOrdered * od.priceEach) AS totalRevenue
    -> FROM products p
    -> JOIN orderdetails od ON p.productCode = od.productCode
    -> GROUP BY p.productCode, p.productName
    -> ORDER BY totalRevenue DESC;
--------------+--------------+
 productName  | totalRevenue |
--------------+--------------+
 Toyota Camry |       390.00 |
 Honda Civic  |       196.00 |
 Yamaha Bike  |        75.00 |
--------------+--------------+
3 rows in set (0.00 sec)
```

25. Find number of customers handled by each employee.

```
ysql> SELECT CONCAT(e.firstName, ' ', e.lastName) AS employeeName,
    ->        COUNT(c.customerNumber) AS totalCustomers
    -> FROM employeess e
    -> JOIN customers c ON e.employeeNumber = c.salesRepEmployeeNumber
    -> GROUP BY e.employeeNumber, employeeName
    -> ORDER BY totalCustomers DESC;
-------------+----------------+
employeeName | totalCustomers |
-------------+----------------+
jane Doe     |              2 |
mike Brown   |              1 |
-------------+----------------+
rows in set (0.00 sec)
```

26. Find customers with more than 5 orders.

```
ysql> SELECT c.customerName, COUNT(o.orderNumber) AS totalOrders
    -> FROM customers c
    -> JOIN orders o ON c.customerNumber = o.customerNumber
    -> GROUP BY c.customerNumber, c.customerName
    -> HAVING COUNT(o.orderNumber) > 5
    -> ORDER BY totalOrders DESC;
Empty set (0.00 sec)
```

27. List products with total quantity ordered > 100.

```
ysql> SELECT p.productName, SUM(od.quantityOrdered) AS totalQty
    -> FROM products p
    -> JOIN orderdetails od ON p.productCode = od.productCode
    -> GROUP BY p.productCode, p.productName
    -> HAVING SUM(od.quantityOrdered) > 100
    -> ORDER BY totalQty DESC;
mpty set (0.00 sec)
```

28. Find employees managing more than 3 customers.

```
ysql> SELECT CONCAT(e.firstName, ' ', e.lastName) AS employeeName,
    ->        COUNT(c.customerNumber) AS totalCustomers
    -> FROM employeess e
    -> JOIN customers c ON e.employeeNumber = c.salesRepEmployeeNumber
    -> GROUP BY e.employeeNumber, employeeName
    -> HAVING COUNT(c.customerNumber) > 3;
Empty set (0.00 sec)
```

29. Show countries with more than 10 customers.

```
mysql> select country , count(*) as customerCount from customers group by country having count(*)>10 order by customerCo
unt desc;
Empty set (0.00 sec)
```

30. Find product lines having more than 5 products.

```
ysql> select productLine,count(*) as totalProducts from products group by productLine having count(*)>5;
mpty set (0.00 sec)

ysql>
```

31. List top 5 customers by total payment amount.

32. Show products ordered by total sales descending.

```
ysql> SELECT p.productName,
    ->        SUM(od.quantityOrdered * od.priceEach) AS totalSales
    -> FROM products p
    -> JOIN orderdetails od ON p.productCode = od.productCode
    -> GROUP BY p.productCode, p.productName
    -> ORDER BY totalSales DESC;
--------------+------------+
 productName  | totalSales |
--------------+------------+
 Toyota Camry |     390.00 |
 Honda Civic  |     196.00 |
 Yamaha Bike  |      75.00 |
--------------+------------+
3 rows in set (0.00 sec)
```

33. List employees sorted by number of customers handled.

```
ysql> SELECT CONCAT(e.firstName, ' ', e.lastName) AS employeeName,
    ->        COUNT(c.customerNumber) AS totalCustomers
    -> FROM employeess e
    -> LEFT JOIN customers c ON e.employeeNumber = c.salesRepEmployeeNumber
    -> GROUP BY e.employeeNumber, employeeName
    -> ORDER BY totalCustomers DESC;
--------------+----------------+
 employeeName | totalCustomers |
--------------+----------------+
 jane Doe     |              2 |
 mike Brown   |              1 |
 john smith   |              0 |
--------------+----------------+
3 rows in set (0.00 sec)
```

34. Show countries sorted by total customers.

```
mysql> select country ,count(*) as totalCustomers from customers group by country order by totalCustomers desc;
---------+----------------+
 country | totalCustomers |
---------+----------------+
 India   |              1 |
 USA     |              1 |
 UK      |              1 |
---------+----------------+
3 rows in set (0.00 sec)
```

35. Display top 10 most ordered products.

Multi-table JOINS

36. Get customer name, order number, and product name.

37. Show employee name, customer name, and total payments.
38. List order details with product line and quantity ordered.
39. Display office city with employee count.
40. Show product vendor with total quantity sold.

Advanced Aggregations

41. Find average order value per customer.
42. Show total revenue generated per country.
43. Find customers whose total payments exceed 50,000.
44. List products where average order quantity > 50.
45. Find top 3 product lines by total revenue.
46. Show customers with credit limit > average credit limit.
47. Find orders where total amount > 10,000.
48. List employees working in offices with more than 5 employees.
49. Find products with stock less than average stock.
50. Show customers who made more than 3 payments.

ADVANCED SQL QUESTIONS (51–80)

Complex Joins + Aggregation

51. Find total revenue generated by each customer (include customer name), sorted by highest revenue.

52. List customers who have placed orders but made **no payments**.

53. Find total number of products ordered by each customer along with total amount spent.

54. Show product-wise revenue and total quantity sold, sorted by revenue descending.

55. Find employees along with total sales (revenue) generated from their customers.

GROUP BY + HAVING (Advanced)

56. Find customers whose total purchase amount is greater than the **average purchase amount of all customers**.

57. List products that have been ordered in **more than 2 different orders**.

58. Find customers who ordered **more than 3 different products**.

59. Show product lines where total revenue exceeds 500.

60. Find countries where average customer credit limit is greater than 60,000.

Multi-level Joins

61. Show customer name, order number, product name, and quantity ordered for all orders.

62. Find employee name, office city, and number of customers handled.

63. Display product line, product name, and total revenue generated.

64. Show order number, customer name, and total order value.

65. List employees whose customers have placed more than 5 orders in total.

Aggregation with Filtering

66. Find customers whose total payments are less than their total order value.

67. List products where total quantity sold is less than average quantity sold across all products.

68. Find customers who have not placed any orders but have made payments.

69. Show employees who have not been assigned any customers.

70. Find orders that contain more than 1 product (multi-line orders).

71. Find top 3 customers based on total order value.

72. Show top 5 products based on total revenue.

73. List top 2 employees based on sales generated.

74. Find top 3 countries by number of customers.

75. Show top 5 most expensive products based on MSRP.

76. Find average order value per customer and list only those above overall average.

77. Show product-wise average price sold (priceEach) vs MSRP difference.

78. Find percentage contribution of each product to total revenue.

79. Calculate average quantity ordered per order for each customer.

80. Find customers whose **largest single order value** exceeds 10,000.